

Backend Engineering Assessment

Multi-Tenant Resource Booking & Availability System

Introduction

Assessment Duration : 24 Hours

This assessment is designed to evaluate your backend engineering skills through the implementation of a multi-tenant resource booking and availability management system.

The primary focus of this assessment is not the number of APIs or features, but rather your ability to:

- Design scalable backend architecture
- Implement clean and maintainable business logic
- Handle multi-tenant data isolation correctly
- Build timezone-aware scheduling logic
- Prevent booking conflicts reliably
- Structure reusable and extensible code
- Demonstrate strong problem-solving ability
- Handle edge cases thoughtfully

The implementation should reflect production-oriented engineering practices.

Project Overview

Develop a backend application where multiple organizations can manage shared resources and allow employees to create time-based bookings.

Each organization should operate in complete isolation with its own:

- Users
- Resources
- Bookings
- Timezone configuration
- Booking policies

The system must support secure tenant-level isolation, role-based access control, booking conflict prevention, and dynamic availability generation.

The availability engine is considered the most important part of this assessment.

Required Technology Stack

The following technologies are mandatory:

- TypeScript
- Node.js
- Express.js
- MongoDB with Mongoose
- Zod for request validation
- Luxon for date/time handling

You may use additional libraries where appropriate.

Core Functional Requirements

1. Authentication & Authorization

Implement a secure authentication system with:

- JWT-based authentication
- Password hashing
- Role-based authorization

Required Roles

- ORG_ADMIN
- EMPLOYEE

Expectations

- Users must only access data belonging to their own organization
- Unauthorized tenant access must be prevented
- Protected route structure should be clean and reusable

2. Organization Management

Each organization should support:

- Name
- Timezone
- Booking configuration
- Active/inactive state

Expectations

- Organization timezone must drive all scheduling logic
- Timezone handling must be implemented correctly using Luxon
- Organization-level isolation must be enforced consistently

3. Resource Management

Organizations should be able to manage shared resources such as:

- Meeting rooms
- Desks
- Devices

Expectations

- Resources belong to a single organization
- Resource names should be unique within an organization
- Resources should support configurable booking buffer times
- Soft delete handling is preferred

4. Booking Management

Employees should be able to create and manage bookings.

Booking Logic Requirements

The system must:

- Prevent overlapping bookings
- Respect resource buffer times
- Respect organization working hours
- Respect organization timezone
- Prevent invalid booking durations
- Prevent bookings outside allowed booking window

Important

Booking conflict detection and scheduling accuracy are important evaluation areas.

5. Availability Engine (Most Important Requirement)

Implement an availability generation system capable of returning valid booking slots dynamically.

The availability calculation must consider:

- Existing bookings
- Resource buffer time
- Booking duration
- Organization timezone
- Organization working hours
- Future booking limits

Expectations

The implementation should:

- Generate accurate available slots
- Handle edge cases correctly
- Be optimized and reusable
- Avoid unnecessary database queries where possible

This module will receive the highest evaluation priority.

Technical Expectations

Your implementation should demonstrate:

- Clean project structure
- Reusable service architecture
- Proper separation of concerns
- Scalable folder organization
- Centralized error handling
- Reusable validation structure
- Strong TypeScript typing
- Query optimization awareness
- Proper MongoDB schema design
- Thoughtful API response handling

Validation Requirements

Validation must be implemented using Zod.

Expected:

- Reusable validation schemas
- Centralized validation handling
- Meaningful validation error responses

Error Handling Expectations

Implement:

- Centralized error middleware
- Consistent error response structure
- Proper HTTP status codes
- Custom error handling where appropriate

Database Expectations

We will evaluate:

- Schema design quality
- Indexing strategy
- Relationship handling
- Query efficiency
- Data isolation strategy

Important Evaluation Areas

The assessment will intentionally evaluate:

- Multi-tenant protection
- Timezone correctness
- Availability calculation logic
- Overlap detection accuracy
- Buffer-time handling
- Booking edge cases

- Architecture quality
- Code maintainability
- Query optimization
- Scalability thinking

Hidden Edge Cases

Your implementation should thoughtfully handle situations such as:

- Back-to-back bookings
- Buffer overlap conflicts
- Midnight or cross-day calculations
- Invalid timezone values
- Duplicate booking attempts
- Concurrent booking requests
- Organization isolation failures
- Invalid booking duration handling

Optional Enhancements (Bonus)

The following are optional but appreciated:

- Swagger/OpenAPI documentation
- Unit or integration testing
- Docker setup
- Request logging
- Rate limiting
- MongoDB transactions
- Environment validation

Submission Requirements

Please submit the following:

- GitHub repository
- Complete README documentation
- Environment variable example file
- API documentation (Swagger/Postman Collection/etc.)
- Setup and run instructions

Mandatory Video Walkthrough

A short walkthrough video is required as part of the submission.

Duration

Approximately 5–15 minutes.

The video should explain:

- Overall project architecture
- Folder structure decisions
- Multi-tenant protection approach
- Availability generation logic
- Booking conflict prevention strategy
- Timezone handling implementation
- Database design decisions
- Assumptions and tradeoffs

The walkthrough video is mandatory.

Evaluation Criteria

Area	Focus
Backend Architecture	Scalability & maintainability
Problem Solving	Logical implementation quality
Multi-Tenancy	Tenant-level isolation
Availability Engine	Slot generation correctness
Luxon Usage	Timezone-aware implementation
MongoDB	Schema & query optimization
Validation	Zod implementation quality
TypeScript	Typing & code safety
Error Handling	Consistency & structure

Code Quality

Readability & maintainability

Edge Cases

Handling completeness

Final Notes

This assessment intentionally prioritizes engineering quality over feature quantity.

We are more interested in:

- clean architecture
- logical correctness
- maintainable code
- scalability thinking
- edge-case handling
- engineering decision-making

than large numbers of APIs or unnecessary features.

Please focus on writing production-oriented backend code.